Game Design Lab 03: Interfaces

V. Markos
Mediterranean College

Spring, 2025 – 2026, Week 06

Abstract

What is this? Probably an ongoing lab for this course, where you will create simple game in Python. Do not expect spectacular graphics and all that stuff, since this course is not about that, but expect to encounter and attack several data structures and algorithm–related problems as we go into more sophisticate designs for our game. Stay tuned for more…

1 Introduction (Or Some First Comments)

Before we actually start with this lab series, some rough comments on what to expect (and what not to) from this lab:

- This is **not an introduction to games design.**
- This is, and should be treated as such, **an introduction to Python programming** and, especially, getting our hands dirty with some actual code, other than converting temperatures, checking primality and all that stuff we see in lab exercises.
- I do not have a full idea of the final version of the game, yet. And, probably, I will not have until Week 12. Many of the features will be decided on the spot, by you and me, as we discuss in class.
- However, I do have a plan on what this game will assess, in terms of data structures and relevant algorithms. Consequently, there are certain features that I have already decided to include.
- However (vol. 2), I also have a certain rough plan for the abstract game outline. There will be a simple game grid, initially covered by a Fog of War, and you will have a player exploring the grid while having to avoid computer–controlled enemies and find some relics which will be collectible and stuff like that — we will further specify features together throughout the next weeks.

2 Artefacts

Last week we designed a quite abstract artefact clas, with the aim of capturing all sort of things that one might be able to collect while playing our game. However, we have some minor issues to address first:

Task 1: Code Refactoring

Last time, due to lack of time, we did not completely transition to using `Artefact` instead of `GridPoint`. So, make any remaining changes to ensure things do work using `Artefact`s.

So far, artefacts are just a thin wrapper around the good old grid points we used to represent our monolithic relics. However, we shall distinguish soon between various artefact types. To begin with, there will be artefacts that will be used to craft our game's ultimate goal; a pitta. Besides that, there will also be artefacts that will be used to craft better collecting equipment, like a sharper higher level knife and so on. So…

Task 2: Artefact Kinds

Having implemnted the parent class `Artefact`, we can now create separate child classes for the various types of artefacts we would like to represent. One might include pitta ingredients, another might be related to equipment, and so on. So, create appropriate child classes to capture all those different kinds of artefacts, taking advantage of inheritance features offered by C#.

3 Inventory

We have yet to implement the `Inventory` class using `Artefact`s and making any required adjustments. So:

Task 3: Inventory Refactoring

Appropriately refactor the `Inventory` class code so as to properly reflect changes / modifications in the inventory class. Utilise polymorphism to allow for a more dynamic behaviour across the various `Artefact` child classes.

Now, what's next? Of course, more refactoring, since Inventories are also used in some place in our codebase. So:

Task 4: Further Reafactoring

Revisit the usage of `Inventory` in `Player` class and wherever we might have used it so far and make sure it is alligned with the new changes made.

4 Interfaces

Artefacts might have different uses, even if they inherit from the same parent class. Moreover, we might want to ensure that any future updates do not break any existing functionalities and also respect a general scheme we might want to use for our game. This is exactly the case where interfaces come in handy! So:

Task 5: Interfaces

Define interfaces for objects that will be used in crafting, such as pitta ingredients, e.g., `Craftable`, and any other interfaces that will determine how certain artefact types will interact with the rest of our game's universe. Then, use those interfaces in the various artefact child classes already implemetned.